UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

Sistemas de información para la gestión – ICN-292

Laboratorio 5

Consultas SQL – Andes Bike

Segundo Semestre 2026

Gonzalo Orellana

Profesor: José Miguel González

Fecha de entrega: 13/11/26

Repositorio GitHub: https://github.com/gorellanap/ICN292-Orellana-Gonzalo

Consulta 1

Código SQL

```
select

	v.nombre as vendedor, count(ven.id_venta) as cantidad_ventas

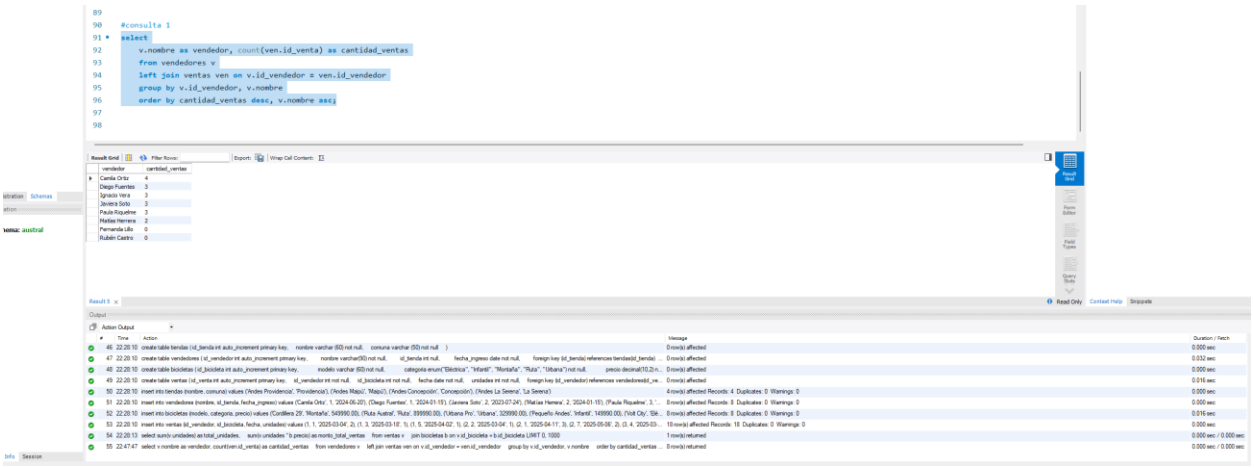
    from vendedores v

    left join ventas ven on v.id_vendedor = ven.id_vendedor

    group by v.id_vendedor, v.nombre

    order by cantidad_ventas desc, v.nombre asc;
```

Captura



Consulta 2

Código SQL

```
select

	t.nombre as nombre_tienda, count(ven.id_venta) as cantidad_ventas,
sum(ven.unidades) as total_unidades

    from tiendas t
```

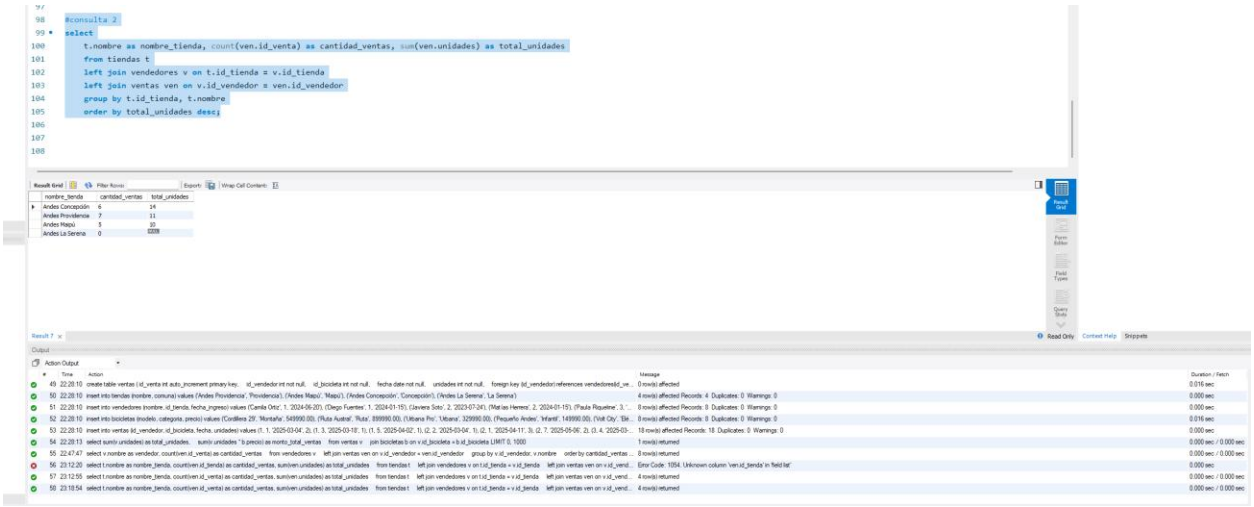
```
    left join vendedores v on t.id_tienda = v.id_tienda

    left join ventas ven on v.id_vendedor = ven.id_vendedor

    group by t.id_tienda, t.nombre

    order by total_unidades desc;
```

Captura



Consulta 3

Código SQL

```
select

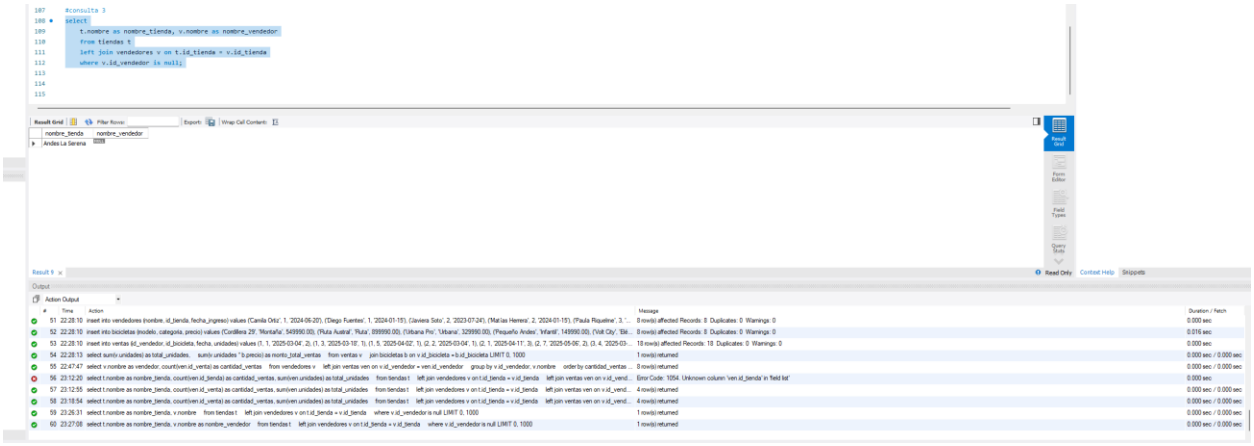
       t.nombre as nombre_tienda, v.nombre as nombre_vendedor

    from tiendas t

    left join vendedores v on t.id_tienda = v.id_tienda

    where v.id_vendedor is null;
```

Captura



Consulta 4

Código SQL

```
b.modelo, b.categoria, ven.unidades

        from bicicletas b

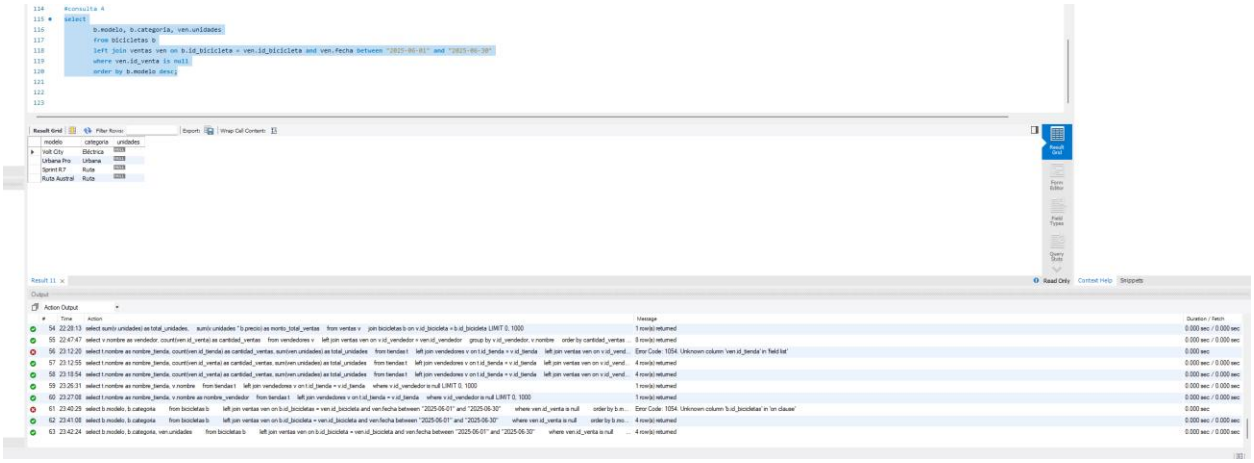
        left join ventas ven on b.id_bicicleta = ven.id_bicicleta and ven.fecha
between "2025-06-01" and "2025-06-30"

        where ven.id_venta is null

        order by b.modelo desc;
```

Explicación: La condición de fecha se colocó en la cláusula ON del left join, para evaluar el filtro antes del emparejamiento. Si se hubiera colocado en el WHERE, habría eliminado las filas con NULL descartando el comportamiento del LEFT JOIN y devolviendo un resultado vacío o incorrecto.

Captura



Consulta 5

Código SQL

```
select

vend.nombre as nombre_vendedor, t.nombre as nombre_tienda, vent.unidades as
unidades_vendidas

        from vendedores vend

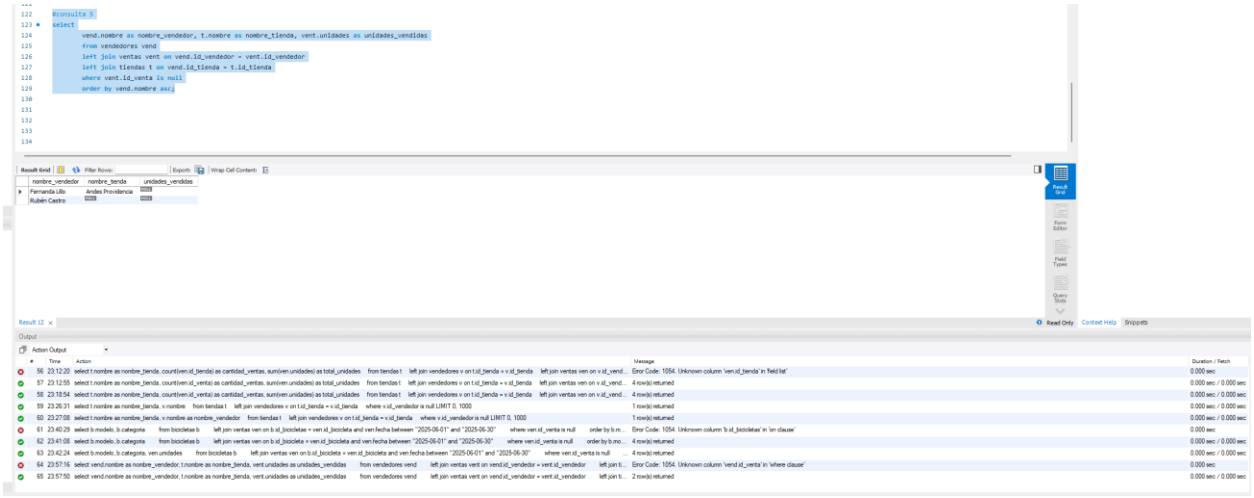
        left join ventas vent on vend.id_vendedor = vent.id_vendedor

        left join tiendas t on vend.id_tienda = t.id_tienda

        where vent.id_venta is null

        order by vend.nombre asc;
```

Captura



Consulta 6

Código SQL

```
select

	vend1.nombre as vendedor_1, vend1.fecha_ingreso as ingreso_1, vend2.nombre as
vendedor_2, vend2.fecha_ingreso as ingreso_2, t.nombre as tienda

from vendedores vend1

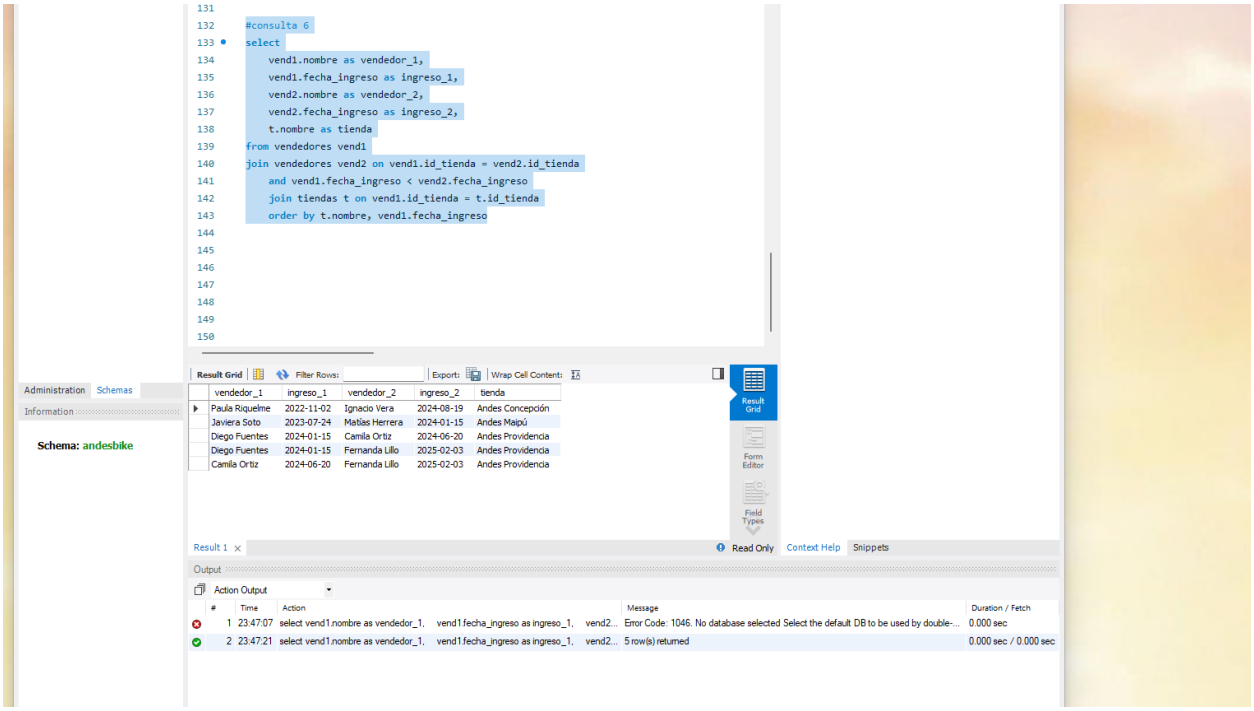
join vendedores vend2 on vend1.id_tienda = vend2.id_tienda

	and vend1.fecha_ingreso < vend2.fecha_ingreso

    join tiendas t on vend1.id_tienda = t.id_tienda

    order by t.nombre, vend1.fecha_ingreso;
```

Captura



Consulta 7

Código SQL

```
select

      t.nombre as tienda, sum(b.precio * ven.unidades) as monto_total

    from tiendas t

    join vendedores vend on t.id_tienda = vend.id_tienda

    join ventas ven on vend.id_vendedor = ven.id_vendedor

    join bicicletas b on ven.id_bicicleta = b.id_bicicleta

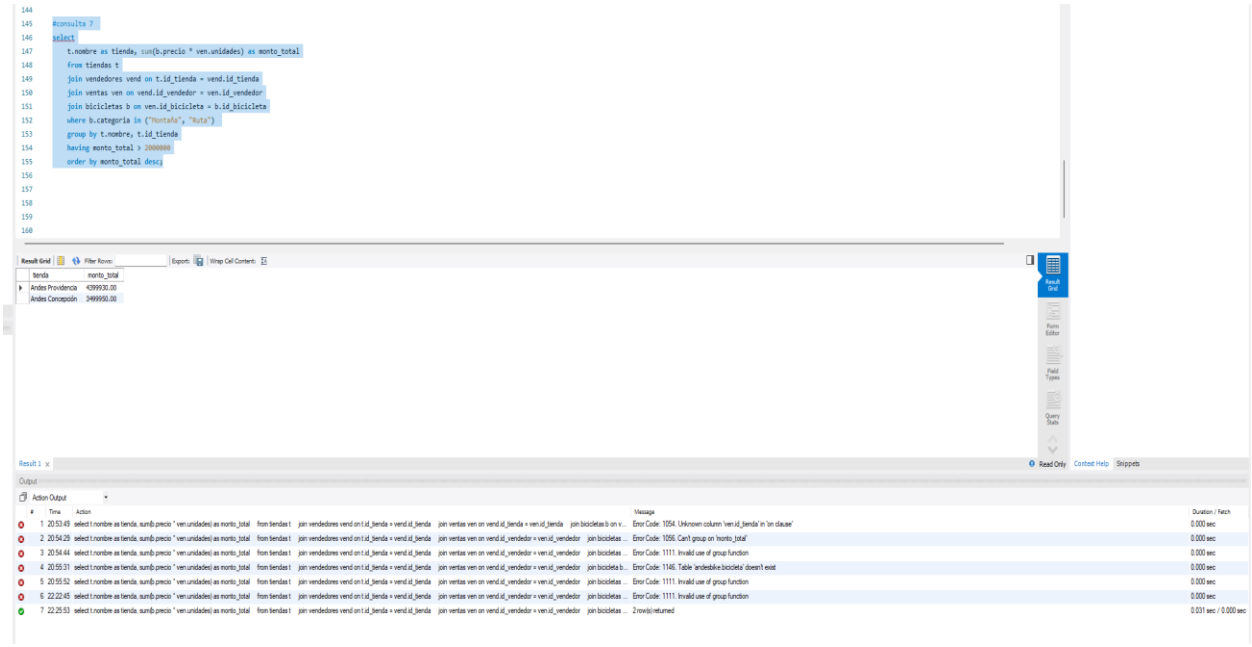
    where b.categoria in ("Montaña", "Ruta")

    group by t.nombre, t.id_tienda

    having monto_total > 2000000
```

```
    order by monto_total desc;
```

Captura



Consulta 8

Código SQL

```
select modelo, precio

from bicicletas

where precio > (select avg(precio) from bicicletas)

order by precio desc;

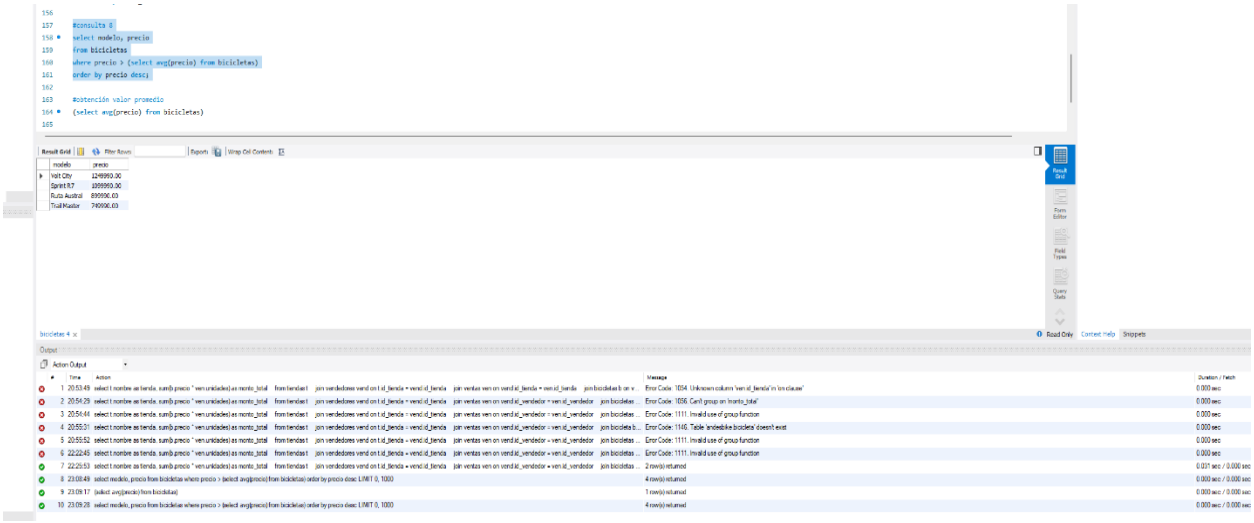


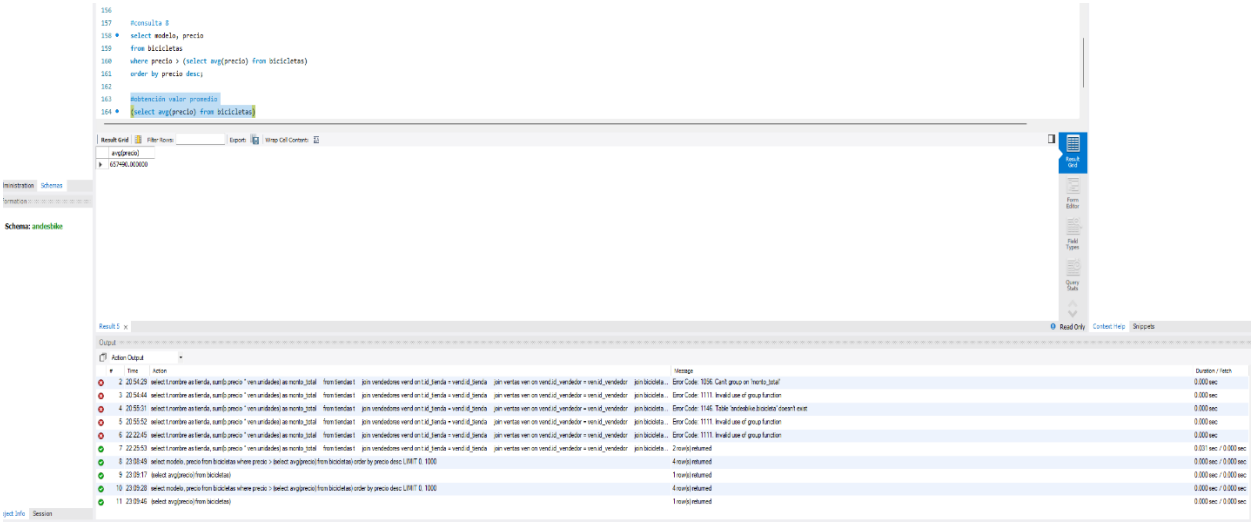
#obtención valor promedio

(select avg(precio) from bicicletas);
```

Explicación: El promedio de precio del catálogo es de **$657.490**. Este valor se obtuvo internamente mediante la subconsulta escalar (SELECT AVG(precio) FROM bicicletas) ejecutada dentro de la cláusula WHERE, lo que permitió a la base de datos calcular el promedio primero y luego filtrar los modelos que lo superaran, devolviendo *Volt City*, *Sprint R7*, *Ruta Austral* y *Trail Master*.

Captura





Consulta 9

Código SQL

```
select

       vend.nombre as vendedor, avg(ven.unidades) as promedio_unidades

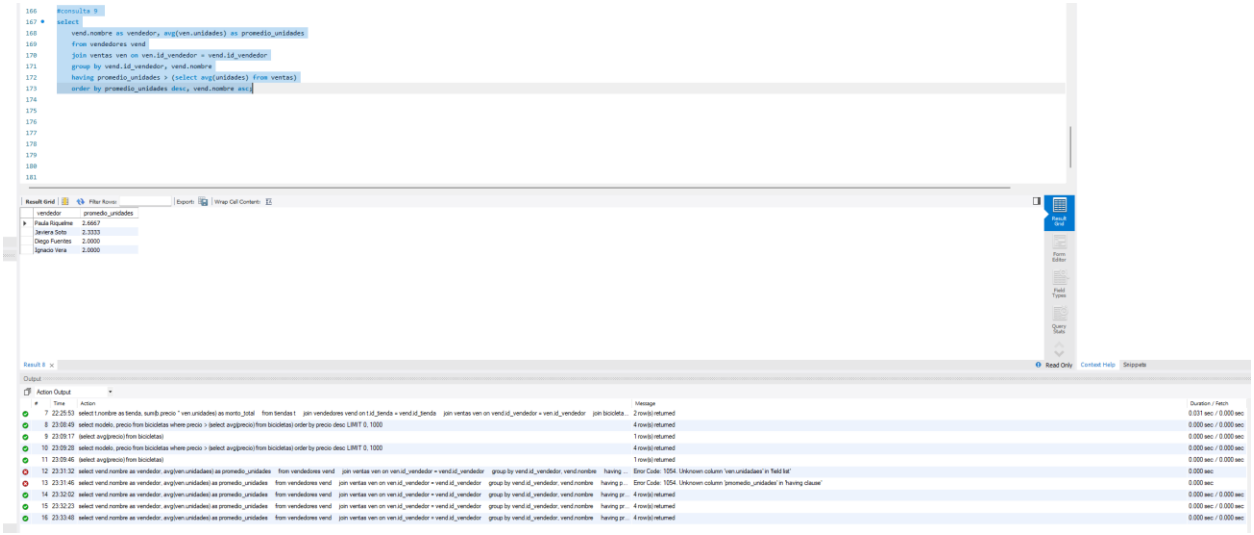
    from vendedores vend

    join ventas ven on ven.id_vendedor = vend.id_vendedor

    group by vend.id_vendedor, vend.nombre

    having promedio_unidades > (select avg(unidades) from ventas)

    order by promedio_unidades desc, vend.nombre asc;Captura
```



Consulta 10

Código SQL

```
select

       vend.nombre as vendedor, sum(ven.unidades * b.precio) as monto_total

    from vendedores vend
```

```
    left join ventas ven on ven.id_vendedor = vend.id_vendedor

    left join bicicletas b on ven.id_bicicleta = b.id_bicicleta

    group by vend.id_vendedor, vend.nombre

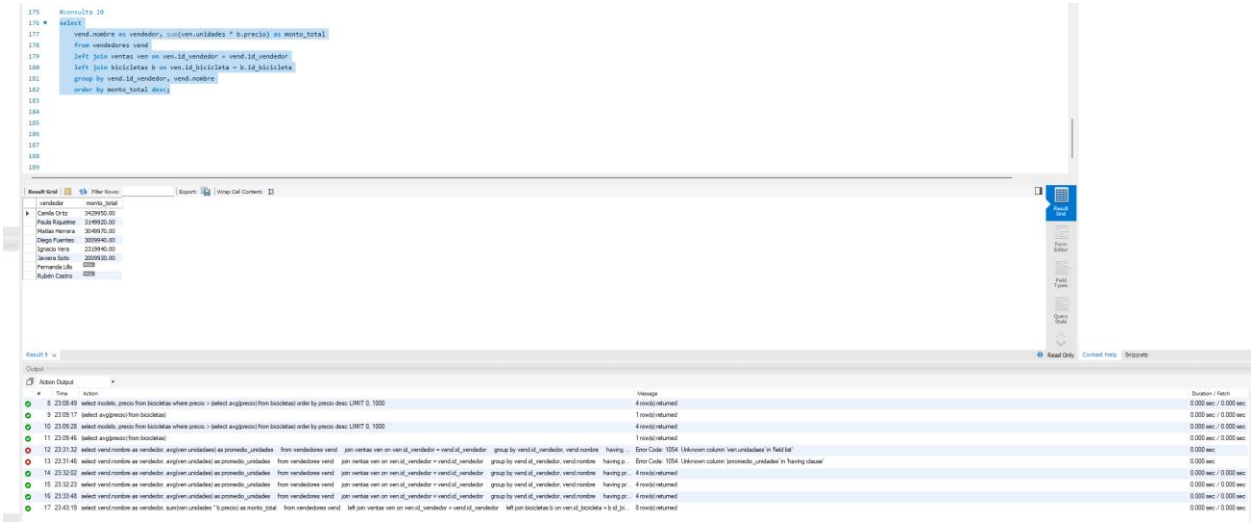
    order by monto_total desc;
```

Algunos vendedores no muestran cero, porque al usar la función sum sobre valores null, vacíos, devuelve el mismo null. Para que se lograra obtener ceros, envés de null, habría que envolver la funciíon sum en otra función que considere los null no como vacio si no como ceros. Por lo que hemos visto en clases y ayudantías, podría usarse la función ifnull() de la siguiente manera:

select

vend.nombre as vendedor,ifnull(sum(ven.unidades * b.precio),0) as monto_total

Captura



Conclusiones

Como conclusión del Laboratorio 5, la aplicación práctica de consultas SQL sobre la base de datos relacional *Andes Bike* permitió consolidar el uso de operadores avanzados de manipulación y análisis de datos en MySQL Workbench. Mediante el uso estratégico de cruces internos y externos (JOIN, LEFT JOIN), técnicas de agregación (GROUP BY, HAVING), y subconsultas escalares, se logró extraer información compleja del negocio, tales como identificación de inactividades comerciales, análisis de rentabilidad por categorías y segmentación de desempeño de vendedores, buscando que siempre se resguarde la integridad lógica de la estructura relacional.